11 • AI 与 GPU 入门

本册是阶段 8 的正文，从 8-01 起步；配套与路线见《17-关卡总表》。

本册共 10 关（8-01 ~ 8-10）。目标：会用 Ollama / vLLM 跑与部署模型，看得懂 nvidia-smi，能复现并排查 CUDA OOM，能把 GPU 指标接进监控，能画全景图讲 10 分钟。

注意：这一册必须在网上跑（Colab / Kaggle）：CUDA 相关命令（nvidia-smi、显存溢出复现）只认 NVIDIA 显卡——**如果你手上是 AMD 卡 / 没有独显，跑不了是正常的，不是你操作错了。**免费资源（Colab / Kaggle）覆盖到绝大多数关卡，不需要买显卡——**唯一例外是 8-08**：那一段要 Docker + NVIDIA 卡，免费档跑不了，书里给了「降级演练」的替代路径。

本册是什么：GPU 与 AI 运维 —— 分清训练与推理、看懂显存、把模型跑起来并接进监控。 用法：按 8-01 ~ 8-10 的编号打关。

动手做实验时照这两条来：实验闭环（先记录原状态 → 操作 → 按输出 / 退出码验证 → 恢复现场）和踩坑记录位（现象 → 原因 → 解决 → 一句话总结）——不必逐关都写，做完整册回头总结一次就够。

8-01 GPU 是谁：训练 vs 推理、显存 vs 内存

这一关不敲命令，只把三组词分清。它们是后面所有关卡的"地图"。

一、训练 vs 推理（最重要的区分）

阶段	在干什么	资源特点	你以后会怎么碰到
训练	拿海量数据反复调整模型参数	几天到几个月、几百张卡、**满负荷运转**	训练任务挂了/变慢了；要重启、要从断点续训
推理	拿训好的模型回答用户	长期在线、**显存吃紧**、要低延迟	推理服务起不来、变慢、显存不够、被限流

一句话记忆：**训练烧算力，推理吃显存**；训练是"批处理"，推理是"在线服务"。AI 运维的日常里，**推理占大头**（因为它在线上跑，随时随地会出问题）。

二、显存 vs 内存（两个不同的"内存"）

```
内存（RAM）  ← 主机/服务器的内存，给 CPU 用；用 ps / free 看
显存（VRAM） ← 显卡自己的内存，给 GPU 用；用 nvidia-smi 看
```

解读：模型参数、中间结果都放在**显存**里。显存不够，程序直接报错（8-04 的 OOM）。**主机内存再大也帮不上显存**——这是新手最容易混的一点。

三、为什么 GPU 比 CPU 快（实战里常被问到）

CPU 像"几个博士"：单个算得很快，但人手少。GPU 像"几千个小学生"：单个算得慢，但**人多、能同时干同一件简单的事。**矩阵运算是"同样的简单计算重复几百万次"，正好是 GPU 的主场 —— 这就是 GPU 在 AI 里不可替代的原因。

完成判定

□ 能用自己的话说清训练和推理的区别，并各举一个运维场景　□ 能用自己的话讲清「这一册学的东西是什么、干什么用的」（见本册开篇）　□ 能说出显存和内存是两回事，各用什么命令看　□ 能用"博士 vs 小学生"讲清 GPU 为什么适合 AI（不掉进数学细节）

回滚方式：纯认知，无操作。

实战价值：工作里常问"你怎么理解 AI 运维和普通运维的区别"，答"推理服务在线上跑、显存是稀缺资源、训练任务要防中断"——三句话就把方向说对了。

8-02 用 Ollama 起一个本地模型并对话

为什么先做这一关：它**零外部依赖**：不抢 GPU、不花钱、断网也能跑（模型下载时除外）。先在自己机器上把"模型跑起来"这件事做成，再谈故障。

步骤 1：装并拉一个小模型

```
$ sudo apt install -y zstd                          # 新装 Ubuntu 没带 zstd，官方脚本解包会报
"requires zstd"
$ curl -fsSL https://ollama.com/install.sh | sh      # 官方一键脚本
$ ollama --version
ollama version is 0.3.14

$ ollama pull qwen2.5:1.5b        # 1.5B 的小模型，约 1GB，纯 CPU 也能跑
pulling manifest ...
success

$ ollama list
NAME              ID            SIZE    MODIFIED
qwen2.5:1.5b      a1b2c3d4e5f6  986 MB  10 seconds ago
```

解读：**1.5B = 15 亿参数**，属于"能塞进普通电脑"的体量。模型文件下载后存在本地（`~/.ollama/models`），之后断网也能用。

官方脚本拉不动？（国内网络常见）：`curl -fsSL https://ollama.com/install.sh | sh` 要连 ollama.com 下载二进制，线路不顺时会 SSL 断流、卡住不装。两条替代路径都免费，选一条：

① **Docker 跑（推荐）**——镜像可以走 7 册「开始之前」配好的加速器：`docker run -d --name ollama -v ollama:/root/.ollama -p 127.0.0.1:11434:11434 ollama/ollama`（**端口安全写法**：`-p 11434:11434` 会监听 0.0.0.0、整个局域网都能访问；写成 `-p 127.0.0.1:11434:11434` 才和原生安装一样只限本机）；之后每个命令加前缀 `docker exec -it ollama`（例：`docker exec -it ollama ollama pull qwen2.5:1.5b`）；"停服务"改 `docker stop ollama`（起回来 `docker start ollama`）。端口 11434、curl 接口与本关其余步骤完全一致；只有第 3 步的 `ss -tulnp | grep ollama` 会查不到——容器路线的监听者显示为 docker-proxy，改 `ss -tulnp | grep 11434` 就行。

② **离线包**：找一台网络顺畅的机器下载 `https://ollama.com/download/ollama-linux-amd64.tgz`，拷到本机执行 `sudo tar -C /usr -xzf ollama-linux-amd64.tgz`；`ollama run` 会自动把服务带起来，"停服务"改 `sudo pkill ollama`。

两条都不通就先跳过安装别硬卡：本关"看端口 / curl 接口 / 停再起服务"的套路，先用 08 册 5-02 起的 Prometheus 当现成靶子练熟，等网络方便再回来补跑模型——8-06 的 `--local` 也一样，等模型就位再做。

步骤 2：对话

```
$ ollama run qwen2.5:1.5b
>>> 用一句话解释什么是运维
运维就是保证线上的服务一直能正常跑、出问题能尽快修好的那批人。
>>> /bye
```

解读：跑起来就像个聊天工具。**但你要关注的不是聊天，是"这东西怎么对外提供服务"**——它默认在本地开了个 HTTP 接口（下一关/下一册就用得上）。

步骤 3：看它开了什么端口（运维视角）

```
$ ss -tulnp | grep ollama
tcp   LISTEN 0 4096  127.0.0.1:11434  0.0.0.0:*  users:(("ollama",pid=3121,fd=3))

$ curl -s http://localhost:11434/api/generate -d '{"model":"qwen2.5:1.5b","prompt":"你
好","stream":false}' | head -c 300
{"model":"qwen2.5:1.5b","created_at":"2026-10-09T22:40:11Z","response":"你好！有什么可以帮你的
吗？",...}
```

解读：三件事：① **11434 是 Ollama 的默认端口**（记住它，就像 9090 之于 Prometheus）；② 它只监听 `127.0.0.1`，所以只能本机访问（这是默认的安全设置）；③ 接口是 HTTP + JSON —— **跟 5-06 拉 Prometheus 指标是同一类活儿。**

步骤 4：制造一次"服务没起来"（提前熟悉故障）

```
$ sudo systemctl stop ollama
$ curl -s http://localhost:11434/api/tags
curl: (7) Failed to connect to localhost port 11434: Connection refused
$ sudo systemctl start ollama && curl -s -o /dev/null -w "%{http_code}\n"
http://localhost:11434/api/tags
200
```

解读：**Connection refused = 端口没人听 = 服务没起来**（对比 4-09 的端口排查链）。`systemctl status ollama` 能看到它为什么停。

完成判定

□ 模型拉下来并能对话（截图或记录一句回答）　□ 能说出模型文件放在哪、默认端口是多少、只听哪个地址
□ 能用 curl 调一次接口拿到 JSON　□ 能故意停掉服务、看到 Connection refused、再恢复

回滚方式：`ollama rm qwen2.5:1.5b` 删模型释放磁盘；`systemctl stop ollama` 停服务。

实战价值："你部署过模型吗"——"我用 Ollama 在本地跑过 1.5B 的小模型，知道它默认 11434 端口、只监听本地、模型文件在哪、怎么用 curl 调它"。这一串已经比大多数转行者强。

8-03 在 Colab 借一张免费 GPU，看懂 nvidia-smi

这一关的目标：不用买显卡，也能在真 GPU 上敲第一条命令，并且**看懂 nvidia-smi 的每一列**——那是 GPU 运维的"仪表盘"。

步骤 1：开一张免费 GPU

```
浏览器打开 https://colab.research.google.com → 新建笔记本
菜单：代码执行程序 → 更改运行时类型 → 硬件加速器选 **GPU（T4）** → 保存
（国内网络优先用 Kaggle（能直连）：https://www.kaggle.com → New Notebook → Settings →
Accelerator 选 GPU，同样免费；Colab 需要能访问 Google 的网络，连不上是网络问题、不是你操作错）

在第一个单元格里输入（Colab 里命令前面加 !）：
!nvidia-smi
```

抢不到卡不是你操作失误：免费 GPU 有额度，高峰时段可能分不到 —— 遇到 "You have exhausted your GPU quota" 或给不到 GPU，换时段（或换 Kaggle）再试即可。

步骤 2：把 nvidia-smi 的每一列读出来

```
+-----------------------------------------------------------------------------------------+
（下面这份输出来自 T4（15360MiB）；换卡型/平台数字会变，读法不变）
| NVIDIA-SMI 550.54.15              Driver Version: 550.54.15      CUDA Version: 12.4     |
|-----------------------------------------+------------------------+----------------------+
| GPU  Name                 Persistence-M | Bus-Id          Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |           Memory-Usage | GPU-Util  Compute M. |
|=========================================+========================+======================|
|   0  Tesla T4                       Off |   00000000:00:04.0 Off |                    0 |
| N/A   42C    P0              26W /   70W |     412MiB /  15360MiB |      0%      Default |
+-----------------------------------------+------------------------+----------------------+
```

看什么	含义 / 什么时候要盯它
GPU 型号（Tesla T4）	不同型号显存和算力差很多；报工单时要写清型号
Driver / CUDA Version	驱动版本决定了能跑什么框架版本 —— 这是环境不匹配的第一嫌疑人
Temp（42C）	温度；长期高温（>80C）要查散热和负载
W / Cap（26W/70W）	功耗与上限；"卡在跑但功耗很低"往往说明它在等数据（见 8-04 延伸）
Memory-Usage（412MiB/15360MiB）	**显存用了多少 / 总共多少**——这一栏是 AI 运维最常盯的数字
GPU-Util（0%）	算力利用率：**高=在算，低=在等**（等数据、等通信、等磁盘）

步骤 3：看看"有人在用"时是什么样

```
!pip install -q torch
import torch, time
print("GPU 可用：", torch.cuda.is_available(), torch.cuda.get_device_name(0))
x = torch.randn(8000, 8000, device="cuda")   # 造一个大矩阵扔进显存
for _ in range(30):
    y = x @ x                                  # 反复做矩阵乘
!nvidia-smi          # 另开一个单元格：看 GPU-Util 和显存的变化
```

解读：跑起来时你会看到 **GPU-Util 飙到 90%+、显存占用涨上去**；跑完又掉下来。**学会"看曲线和数字判断它在干活还是在等"**，就是 GPU 运维的基本功。

步骤 4：记录一次环境信息（这是产出）

```
# 把这三样抄进你的笔记（以后报工单/排查都要写）
1. GPU 型号：Tesla T4
2. 驱动 + CUDA 版本：550.54.15 / 12.4
3. PyTorch 版本与是否能看到 GPU：torch 2.x / True
```

完成判定

□ 能在 Colab（或 Kaggle）里拿到 GPU 并跑出 `nvidia-smi` □ 能逐列说出含义（至少：型号、驱动/CUDA、温度、功耗、显存、利用率） □ 能说出 GPU-Util 高和低各说明什么 □ 能记录一次完整环境信息 □ 知道抢不到卡时该怎么办

回滚方式：Colab 会话关掉即回收；不留本机痕迹。

实战价值：工作里常问"你怎么判断一张卡是不是在干活"，答"看 nvidia-smi 的 GPU-Util 和功耗：利用率高是在算，利用率低说明在等数据或卡上没任务"——这已经是有经验的答法了。

8-04 故意把显存撑爆：复现 CUDA out of memory（本册心脏）

为什么它是心脏：这类岗位更看重你面对没见过的故障，而不是背诵。**OOM（显存溢出）是 AI 运维最高频、也最能讲出故事的故障**——你要亲手把它造出来，读懂那段报错，再想出至少四种缓解办法。

环境兜底（抢不到免费卡也能完成本关）：本关要在真 GPU 上复现 OOM；**配额用尽、平台排队时别干等**——

① **同方法降级演练（推荐）**：用 `ulimit -v` 先把试炼进程的内存上限压小、再申请超大内存，得到干净的 `MemoryError`：`bash -c "ulimit -v 500000; python3 -c 'x = bytearray(20 * 1024**3)'"`；想经历"被系统 OOM 杀"就用限内存容器（有 Docker 时）：`docker run --memory=256m python:3.12 python3 -c 'x = bytearray(20 * 1024**3)'`（退出码 137 = OOMKilled）——**用同一套"现象 → 线索 → 建议"三层读法逐句拆**，方法与 CUDA OOM 完全一致，只是换了故障现场。**WSL 用户特别注意：别在本机直接"申请超大内存"赌系统杀进程——实测会把整个 WSL 发行版打挂（连接断开、容器与 k3s 全断）；要么像上面用 `ulimit -v` 限住，要么把演示关进限内存容器。**

② 记录里如实标注："**免费额度受限，未实测 CUDA，用内存 OOM 做了同方法演练**"；

③ 对外就按记录说："CUDA OOM 我先用内存 OOM 做了同方法演练，三层读法一致；拿到卡 30 分钟能复现真版。"——如实说反而加分，硬吹最容易被追问拆穿。

步骤 1：把显存吃满

```
import torch
size = 20000                          # 先试 20000 × 20000
x = torch.randn(size, size, device="cuda")

RuntimeError: CUDA out of memory. Tried to allocate 1.49 GiB.
GPU 0 has a total capacity of 14.56 GiB of which 13.94 GiB is free.
Of the allocated memory 0 bytes is allocated by PyTorch,
and 12.31 MiB is reserved by PyTorch but unallocated.
If reserved but unallocated memory is large try setting
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.
```

（这段是 PyTorch 的 CUDA OOM 报错；**核心几句与新版本逐字一致**，只有末尾 `expandable_segments` 那句的措辞新版略有改写——以你机器上的报错为准，读法不变。）

照做复现不了？上面这段报错来自一台**显存已被占用**的机器（示例）。在全新 Colab / Kaggle 的 T4（约 13.9 GiB 空闲）上，`size = 20000`（只申请 1.49 GiB）**根本不会崩**——把 `size` 继续加大到约 **6.2 万**（$4 \times 62000^2 \approx 14.3$ GiB）才会 OOM；想稳定复现也可以先占住一部分显存再跑。

逐句读这段报错（这就是"看懂报错"）：

原句	它在说什么
`Tried to allocate 1.49 GiB`	**现象**：这一次想申请 1.49G
`total capacity of 14.56 GiB of which 13.94 GiB is free`	**线索**：卡上还有 13.94G 空闲 —— 不是"卡太小"：要么这次申请的块特别大，要么是**碎片 / 别的进程占着**把显存掏出了窟窿
`0 bytes is allocated by PyTorch`	**线索**：PyTorch 自己还没占多少，说明是这一步的尺寸要得太狠
`try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`	**建议**：官方给的缓解办法之一（治碎片化）

读报错三步法（读一行报错）：先看**现象**（要了多少、在哪失败）→ 再看**线索**（还剩多少、谁占的）→ 最后看**建议**（官方解法常常就写在最后几行）。14 册《日志报错解读》讲的就是这个顺序。

步骤 2：四种缓解办法，一个一个试

```
# 办法 1：把尺寸改小（最直接）
size = 12000
x = torch.randn(size, size, device="cuda")
print("占用：", round(torch.cuda.memory_allocated() / 1024**3, 2), "GiB")

# 办法 2：用完就释放（避免"垃圾"占着显存）
del x
torch.cuda.empty_cache()

# 办法 3：数据分批（batch 变小）—— 训练里最常用
# 原来一次喂 64 条 → 改成 16 条，显存占用大致降到 1/4

# 办法 4：治碎片化（报错里建议的那个环境变量）
# 注意：它只在"启动前"生效—写成启动命令的一部分最稳：
#   PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True python train.py
# 已经跑起来的会话里再改下面这行 os.environ，不会重新生效（要重启进程）
import os
os.environ["PYTORCH_CUDA_ALLOC_CONF"] = "expandable_segments:True"
```

解读：四种办法对应四种病因 —— **要得太多（尺寸）/ 没及时释放（泄漏）/ 一次喂太多（batch）/ 内存碎片（碎片化）**。工作里常问"OOM 怎么处理"，按这四类答，就不会只答"换大卡"。

步骤 3：复现"明明还有显存却还是 OOM"

```
a = torch.randn(4000, 4000, device="cuda")   # 4000² × 4B = 64 MB ≈ 0.06 GiB
b = torch.randn(4000, 4000, device="cuda")
del a
b = b * 1.0                                   # 制造一些碎片
try:
    c = torch.randn(3000, 3000, device="cuda")
    print("这次成功")
except RuntimeError as e:
    print("失败：", str(e).splitlines()[0])
torch.cuda.empty_cache()                      # ← 关键动作
print("清理后：", round(torch.cuda.memory_reserved() / 1024**3, 2), "GiB 仍被保留")
```

解读：这就是报错里那句 "reserved but unallocated" 的真实含义 —— **显存被 PyTorch 抓着但没用上**。看到这个关键词，第一反应就是 `empty_cache()` 或调 batch。

（这段是"演示"不是"必修"：这套 64MB 级的张量在 15GB 的 T4 上跑，**大概率打印"这次成功"**——碎片型 OOM 要接近满载时才会出现，小尺寸看到成功属正常；关键是理解 `reserved` 与 `allocated` 的差别。）

步骤 4：把这次排查写成记录（产出）

```
# 显存溢出排查记录
现象：报 CUDA out of memory，Tried to allocate X GiB
线索：报错里"还剩多少空闲"最关键 —— 剩得多说明要得太狠或碎片化；剩得少说明真不够
四类病因与对策：
  1) 尺寸/模型太大     → 减小输入、换更小模型、量化（参数用 8-bit 等低精度存，省显存）
  2) 没及时释放        → del + torch.cuda.empty_cache()
  3) 一次喂太多        → batch_size 调小、梯度累积
  4) 内存碎片          → PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True
验证：改完后跑通，并打印 memory_allocated() 确认占用在预期范围
一句话总结：OOM 先读"还剩多少"，再按四类病因对号入座
```

完成判定

□ 亲手复现一次 CUDA out of memory（记录报错原文；抢不到卡时按"环境兜底"用内存 OOM 做同方法演练并如实标注） □ 能逐句读出报错里的"现象 / 线索 / 建议" □ 能说出四种缓解办法及各自对应的病因 □ 写出这份排查记录 □ 能说出 `reserved but unallocated` 是什么意思

回滚方式：Colab/Kaggle 里"运行时 → 重新启动会话"即释放全部显存；本机跑就杀掉进程。

实战价值：这是整册最能讲的一关 —— "我故意复现过 CUDA OOM，能读懂报错的三层信息，并准备了四类缓解方案"。这是踩过的坑，不是背的文档（没抢到卡：按上方"环境兜底"如实说明口径）。

延伸：训练变慢了怎么拆（生产里最典型的"模糊故障"）

为什么单独讲它：前面几关的故障都是"坏了"（起不来、连不上）。而真实工作里更常见的是"没坏，但慢了"——没有报错、没有红灯，只有一句"今天训练怎么这么慢"。**这类故障考的就是"把模糊问题拆成可验证的假设"**，正是实际工作里最需要的能力。

分层拆法（从上到下，一层一层看）：

层	先看什么	慢了通常说明什么
① **GPU 计算**	`nvidia-smi` 的 GPU-Util 是不是长期接近 100%	长期 100% = 计算是瓶颈（正常情况）；**很低** = GPU 在等数据/等通信，问题在下面几层
② **显存**	显存占用是否已接近上限、有没有 swap / OOM 前兆	显存吃满会让框架反复腾挪、变慢（8-04 的 OOM 是它的极端形态）
③ **通信（多卡）**	NCCL 日志里各卡等待时间是否拉长	某张卡拖后腿（慢卡/坏卡）会让所有卡一起等——木桶效应
④ **网卡 / 网络**	多机训练时网卡流量、是否有丢包/重传	参数同步要走网络；网络抖动直接表现为"训练一步忽快忽慢"
⑤ **存储 / 读数据**	数据盘 IO、checkpoint 目录的写入速度	checkpoint 写满带宽时，训练会明显卡顿（这就是为什么常看到"每存一次 checkpoint 就慢一下"）
⑥ **数据加载**	DataLoader 的 worker 数、CPU 是否打满	读数据跟不上 GPU：GPU 利用率低 + CPU 高，最典型的"饿着显卡"

怎么用它（假设 → 验证）：

```
现象：训练比昨天慢了一倍
假设 1：GPU 在等数据   → 验证：nvidia-smi 里 GPU-Util 只有 20%、CPU 打满 → 成立：加大 DataLoader worker
假设 2：某张卡拖后腿   → 验证：nvidia-smi 里 8 张卡里 1 张显存/利用率异常 → 成立：报修/换卡，先把它排除
假设 3：checkpoint 拖慢 → 验证：把保存间隔调长再跑一段，速度回升 → 成立：把 checkpoint 放到更快的盘 / 错峰
```

一句话记住：**先分清是"GPU 在干活"还是"GPU 在等"** —— 在干活（利用率高）说明瓶颈可能在计算本身；在等，就往"数据 / 通信 / 存储"三层去找。

实战价值：别人问"训练变慢你怎么查"，能答出这张分层表 + "我先看 GPU 利用率再决定往哪层走"的人，就是在用运维的思维做 AI 运维。

顺带一提 • 断点续训（checkpoint / 断点续训）
是什么：训练过程中定期把"模型参数 + 优化器状态 + 走到第几步"存成文件（checkpoint）；万一机器挂了或者要停机维护，从最近一次 checkpoint 接着练，不用从头来。
为什么大模型必须要它：一次训练要跑几天到几个月、占用成百上千张卡 —— 中途不出任何问题几乎不可能。**没有它，一次断电就等于烧掉几百万。**所以"能不能从断点恢复"是大模型训练的生死线。
真要用时去哪查：搜"PyTorch checkpoint resume training"或"deepspeed save_checkpoint"；顺手记一句：**存 checkpoint 会抢磁盘带宽，所以经常能看到"每存一次就慢一下"**（这正是上面那张分层表第 ⑤ 层）。
顺带一提 • NCCL 通信延迟抖动 与 checkpoint 耗时
NCCL 是什么：多张卡之间"对答案"用的通信库（集合通信）。各卡算完一批梯度要互相汇总，谁慢了大家都得等——所以叫"通信延迟抖动"。
为什么要盯它：训练忽快忽慢、多卡利用率参差不齐，很多时候根因就在通信上（某张卡慢、某条链路抖），而不是模型的问题。**生产里的日常，盯的就是它们。**
checkpoint 耗时：写一次 checkpoint 会同时吃掉存储带宽，短则几秒、长则几十秒，期间训练几乎停顿 —— 高频保存会明显拉低整体吞吐。
真要用时去哪查：搜"NCCL debug log"（设 `NCCL_DEBUG=INFO` 看每步耗时）和"checkpoint 加速"（异步保存 / 更快的数据盘）；先知道有这两个东西、知道它们会拖慢训练，就够这一轮用了。

8-05 跑一个最小训练

为什么只跑两轮：你不需要学会训练模型 —— **只需要"跑过一次真的训练"，这样 8-04 的显存问题才有具体场景**，聊起来也能说“训练和推理我都见过”。所以就用最小的 MNIST，两轮收工。

步骤 1：两轮训练跑起来

```
import torch
from torch import nn
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

tf = transforms.ToTensor()
train = datasets.MNIST(root="./data", train=True, download=True, transform=tf)
loader = DataLoader(train, batch_size=64, shuffle=True)

model = nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Linear(128,
10)).cuda()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
loss_fn = nn.CrossEntropyLoss()

for epoch in range(2):                        # 就两轮
    for i, (x, y) in enumerate(loader):
        x, y = x.cuda(), y.cuda()
        loss = loss_fn(model(x), y)
        opt.zero_grad(); loss.backward(); opt.step()
        if i % 200 == 0:
            print(f"epoch {epoch} step {i} loss {loss.item():.3f}")

epoch 0 step 0 loss 2.315
epoch 0 step 200 loss 0.421
...
epoch 1 step 800 loss 0.187
```

解读：**loss 在往下走 = 训练在正常进行**（2.3 → 0.19）。这一步只要跑得通、看得懂 loss 下降，就达标了。

步骤 2：和显存对上号（把两关连起来）

```
print("batch_size=64 时显存占用：", round(torch.cuda.max_memory_allocated()/1024**2, 1),
"MiB")

torch.cuda.reset_peak_memory_stats()          # ← 清掉上一次的峰值，否则第二次读到的还是历史最
高

# 把 batch_size 改成 256 再跑一次，看占用变成多少：
loader2 = DataLoader(train, batch_size=256, shuffle=True)
（重复上面的训练循环，只跑 50 步）
print("batch_size=256 时显存占用：", round(torch.cuda.max_memory_allocated()/1024**2, 1),
"MiB")
```

（这个 MLP 很小，峰值只有**几十 MiB**——用 GiB 两位小数会显示成 0.0，所以这里改用 MiB；两次之间要 `reset_peak_memory_stats()` 才看得出变化。）

解读：**batch 越大、显存占用越大**（大致成正比）。这就是 8-04 里"办法 3：数据分批"的现实依据 —— 显存不够时，把 batch 调小是训练侧最常用的手法。

完成判定

□ 两轮训练跑通，loss 明显下降　□ 能说出 loss 下降说明什么　□ 用两个不同 batch_size 各跑一次，说出显存占用的变化趋势　□ 能说出"batch 大小 → 显存 → OOM"这条因果链

回滚方式：Colab 会话重启即清；下载的 MNIST 数据在 `./data`，删掉即可。

实战价值：工作里常问"你了解训练吗"，能说"我跑过最小训练，知道 loss 怎么看、batch 和显存的关系"，就不会被当成"只会用工具的人"。

8-06 用 Python 调 DeepSeek API 做命令行助手

这一关的产出是一个你能天天用、也能拿去讲的小工具：命令行里敲一行，模型回答并记进日志。它把阶段 3 的 requests / argparse / logging 和阶段 8 的"模型服务"接在一起。

步骤 1：拿一个 API Key（没有也能做完）

```
1) 注册 https://platform.deepseek.com → API Keys → 新建，复制（只显示一次）
2) 不要写进代码！放进环境变量：
   $ echo 'export DEEPSEEK_API_KEY="sk-你的key"' >> ~/.bashrc && source ~/.bashrc
3) 验证：$ echo $DEEPSEEK_API_KEY | head -c 6
4) 要充值：按量计费（几块钱够练很久）；新账号不一定有免费额度，没余额调用会被拒
5) 看到 402 Insufficient Balance 别慌：那不是 key 配错，是余额不足，充值后即恢复

# 不想充钱也行：用 8-02 的本地 Ollama 当"备胎"（下面脚本加 --local，两种后端都支持）
```

安全红线：Key 是钱，**绝不提交进 GitHub**。提交前用 `git status` 看一眼，必要时在 `.gitignore` 里加 `.env`、`.env.*`、`*.env` 三行（和 16 册的写法一致）。

步骤 2：脚本（两种后端都支持，可直接复制）

```
#!/usr/bin/env python3
"""命令行小助手：--ask "问题"；默认走 DeepSeek，--local 走本地 Ollama"""
import argparse
import json
import logging
import os
import sys

import requests

DEEPSEEK = "https://api.deepseek.com/chat/completions"
OLLAMA = "http://localhost:11434/api/generate"


def ask_deepseek(prompt, timeout=30):
    key = os.environ.get("DEEPSEEK_API_KEY")
    if not key:
        raise RuntimeError("没有找到 DEEPSEEK_API_KEY（见步骤 1）")
    resp = requests.post(
        DEEPSEEK,
        headers={"Authorization": f"Bearer {key}", "Content-Type": "application/json"},
        json={"model": "deepseek-chat",
              "messages": [{"role": "user", "content": prompt}]},
        timeout=timeout,
    )
    resp.raise_for_status()
    data = resp.json()
    return data["choices"][0]["message"]["content"]


def ask_local(prompt, timeout=120):
    resp = requests.post(OLLAMA, json={"model": "qwen2.5:1.5b",
                                       "prompt": prompt, "stream": False}, timeout=timeout)
    resp.raise_for_status()
    return resp.json()["response"]


def main():
    p = argparse.ArgumentParser(description="命令行小助手")
    p.add_argument("--ask", required=True, help="要问的问题")
    p.add_argument("--local", action="store_true", help="改用本地 Ollama")
    p.add_argument("--timeout", type=int, default=30)
    args = p.parse_args()

    logging.basicConfig(filename=os.path.expanduser("~/assistant.log"), level=logging.INFO,
                        format="%(asctime)s %(levelname)s %(message)s")
    try:
        answer = ask_local(args.ask, args.timeout) if args.local else ask_deepseek(args.ask,
args.timeout)
    except Exception as exc:
        logging.error("调用失败：%s", exc)
        print(f"[错误] {exc}", file=sys.stderr)
        return 1
    logging.info("问：%s | 答：%s", args.ask, answer[:80])
    print(answer)
```

```
    return 0


if __name__ == "__main__":
    sys.exit(main())
```

解读（三处关键）：① **key 从环境变量读**，不写进代码；② **两种后端用一个开关切换**——没 key 时用 `--local` 照样能跑通，这就是 8-02 那一关的价值；③ 异常与日志走 3-09 的套路，退出码 0/1。

步骤 3：跑三种情况

```
$ python3 assistant.py --ask "用一句话解释什么是显存"
显存是显卡自带的内存，专门存放模型和数据，不够就会报 CUDA out of memory。

$ python3 assistant.py --local --ask "你好"
你好！有什么可以帮你的吗？

$ DEEPSEEK_API_KEY="" python3 assistant.py --ask "测试"
[错误] 没有找到 DEEPSEEK_API_KEY（见步骤 1）          ← 报错不乱
$ tail -2 ~/assistant.log
2026-10-09 23:05:11,201 INFO 问：用一句话解释什么是显存 | 答：显存是显卡自带的内存…
```

完成判定

□ 脚本能带 `--ask` 提问并打印回答　□ `--local` 能切到本地模型（没 key 也能演示）　□ 出错时给中文提示、报错不乱、日志有记录　□ key 没有出现在代码里　□ 能说出"为什么 key 要放环境变量"

回滚方式：删掉脚本与日志即可；key 在环境变量里，随时可在控制台吊销。

实战价值：这是"你写过什么脚本"的第二个答案，而且它证明你会**同时管两种模型服务**（云 API 和本地推理），还知道密钥不能进仓库。

8-07 用 vLLM 部署一个能访问的推理服务

为什么要 vLLM：Ollama 适合"自己玩玩"，而**生产上跑推理服务常用 vLLM**——它专为高并发推理优化。这一关让你从"跑模型"升级到"**部署一个服务**"：有端口、有接口、有日志、能排障。

环境提示：vLLM 需要 Linux + NVIDIA 显卡 + CUDA。**没有显卡怎么办**：① Colab / Kaggle 免费 GPU 上 `pip install vllm` 跑（推荐；国内网络优先 Kaggle——Colab 需要能访问 Google 的网络，详见 8-03 的说明）；② 只把配置和命令读一遍、在 Ollama 上做等价的"服务化"练习（8-02 已经做过一次）。**两条路都算过关**；记录里标清口径：真卡跑的写"实测"，走等价练习的写"未实测 CUDA，已用 Ollama 等价演练"——对外就按记录说，不夸张。

步骤 1：起服务（Colab 或本机 GPU 机器）

```
$ pip install -q vllm
$ python -m vllm.entrypoints.openai.api_server \
    --model Qwen/Qwen2.5-1.5B-Instruct \
    --port 8000 --gpu-memory-utilization 0.8 &
INFO: Started server process [1234]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000
```

解读：三个参数要记住 —— `--model`（用哪个模型）、`--port`（听哪个端口）、`--gpu-memory-utilization 0.8`（**最多用 80% 显存**：留一点给别的进程，也是防 OOM 的常用手段，和 8-04 呼应）。**最后一行"Uvicorn running"= 起来了。**

步骤 2：验证它能用

```
$ curl -s http://localhost:8000/v1/models | head -c 200
{"object":"list","data":[{"id":"Qwen/Qwen2.5-1.5B-Instruct","object":"model",...}]}

$ curl -s http://localhost:8000/v1/chat/completions -H "Content-Type: application/json" -d '{
  "model": "Qwen/Qwen2.5-1.5B-Instruct",
  "messages": [{"role": "user", "content": "用一句话说明什么是推理服务"}]
}' | head -c 300
{"id":"chatcmpl-...","choices":[{"message":{"role":"assistant","content":"推理服务是把训练好的
模型部署成接口，让用户或程序随时调用。"}}]}
```

解读：**vLLM 提供的接口和 OpenAI 的格式一样**（`/v1/chat/completions`）—— 这意味着你 8-06 的脚本只要把 URL 换一下就能指向自己的服务。这是行业惯例，实战里也常见。

步骤 3：故意让它起不来，然后排障（三种原因）

```
# ① 端口被占（最常见）
$ python -m vllm.entrypoints.openai.api_server --model Qwen/Qwen2.5-1.5B-Instruct --port 8000
OSError: [Errno 98] Address already in use
→ 排查：ss -tulnp | grep 8000     → 处理：换端口（--port 8001）或停掉占用者

# ② 显存不够
ValueError: The model's max seq len (32768) is larger than the maximum number of tokens
that can be stored in KV cache. Try decreasing max_model_len.
（文案随 vLLM 版本变化：新版写成 "… Try increasing gpu_memory_utilization … or decreasing
max_model_len …"——认 KV cache / max_model_len / gpu_memory_utilization 这几个关键词就够）
→ 排查：nvidia-smi 看还剩多少显存      → 处理：加 --max-model-len 4096 或调小 --gpu-memory-
utilization

# ③ 模型路径/名字写错
OSError: Can't load the configuration of 'Qwen/Qwen2.5-1.5B'
→ 排查：确认模型名拼写、有没有下载权限      → 处理：改回正确名字
```

解读：**"端口 / 显存 / 模型路径"就是推理服务起不来的三大类原因**——和 4-09 的容器排障是同一套思路，只是换了对象。

步骤 4：把它当服务来管（这才是运维的部分）

```
$ ss -tulnp | grep 8000            # 确认谁在听
$ tail -f vllm.log                 # 盯日志（别只看启动那几行，要看请求进来时的输出）
$ nvidia-smi                       # 另开一个终端盯显存和利用率
```

解读：**起服务只是第一步，能"看着它跑、出问题能查"才是运维。**这也是行业里常说的“黑盒变透明”。

完成判定

□ 服务起得来，`/v1/models` 能返回模型列表　□ 能用 curl 发一次请求拿到回答　□ 能说出 `--gpu-memory-utilization` 为什么要留余量　□ 能说出推理服务起不来的三类原因及各自的排查命令　□ 能说清 vLLM 的接口为什么和 OpenAI 一样

回滚方式：`kill %1`（或 kill 掉进程）停服务；Colab 会话关掉即回收。

实战价值："你部署过推理服务吗"——答"我用 vLLM 起过服务，知道它的接口和 OpenAI 兼容，也知道起不来通常是端口、显存或模型路径三类问题"。这是**直接来自实际工作的说法**。

8-08 把 GPU 指标接进监控面板

把两条线接起来：8 册你搭了 Prometheus + Grafana，但那时面板上只有 CPU / 内存 / 磁盘。**这一关把 GPU 也接上去**——"黑盒变透明"，这也正是行业里的常见说法。

步骤 1：GPU 指标要有个"翻译官"

```
node-exporter 管 CPU/内存/磁盘，但它**看不到 GPU**。
GPU 指标要用 dcgm-exporter（NVIDIA 官方）或 nvidia_gpu_exporter。

# 需要一台能跑 docker --gpus 的 NVIDIA 机器（本机 N 卡 / 按需租的云 GPU）；注意：Colab、Kaggle 免费档没有 Docker，跑不了这段
$ docker run -d --name dcgm --gpus all -p 9400:9400 nvcr.io/nvidia/k8s/dcgm-exporter:latest
（镜像 tag 随 NVIDIA 发布更新—这里用 latest 最省心；想固定版本就去 dcgm-exporter 的 release notes 挑一个确实存在的 tag，写错 tag 会拉不到镜像。）
$ curl -s http://localhost:9400/metrics | grep -m3 DCGM_FI_DEV
DCGM_FI_DEV_GPU_UTIL{gpu="0",...} 0
DCGM_FI_DEV_FB_USED{gpu="0",...} 412
DCGM_FI_DEV_FB_FREE{gpu="0",...} 14948
```

环境兜底（没有卡也能过关）：这一关的完整版需要"能跑 `docker --gpus all` 的 NVIDIA 机器"——**这是本册唯一一处免费资源搞不定完整版的地方**（Colab / Kaggle 免费档有 N 卡但没 Docker，只有按小时付费的云 GPU 才能真跑这段）。两条替代路径，选一条：

① **降级演练（推荐）**：在 Colab / Kaggle 跑 `nvidia-smi`，把输出整理成一小段"指标文本"，让 Prometheus 以文件方式读它——**实操要点（实测）**：指标文本存成 `.txt` 这类带正确类型的后缀（`python3 -m http.server` 对无后缀 / 未知后缀的文件会发 `application/octet-stream`，Prometheus 拒收 `received unsupported Content-Type`；.txt 实测即 UP）；步骤 2 的 job 配置照写、**加一行 `metrics_path: /gpu.txt`**、targets 换成本机固定端口——**面板结构、PromQL、Targets UP 都能练通**；截图标注"环境限制，未实测"。

② **有 N 卡机器**（本机 / 借用 / 按需租）：装好驱动 + nvidia-container-toolkit，按原步骤真接一遍。

两条路都请读完步骤 2~4：配置与 PromQL 才是本关要考的。

解读：`DCGM_FI_DEV_GPU_UTIL` = 利用率，`FB_USED` / `FB_FREE` = 显存已用 / 剩余（单位 MiB）。**这些数字就是 nvidia-smi 背后读的东西**——现在它们变成了可以让 Prometheus 抓的接口。

步骤 2：让 Prometheus 抓它

```
$ cd ~/lab/monitor
$ nano prometheus.yml     # 在已有 scrape_configs: 下追加下面这一段—别整份覆盖（原有的 node、10 册加的 job 会被冲掉）
  - job_name: "gpu"                              # ← 新增这一段（缩进对齐上一个 job）
    static_configs:
      - targets: ["host.docker.internal:9400"]
# 并在 docker-compose.yml 的 prometheus 服务下加（Linux / WSL 原生 Docker 必须加，
# 否则 host.docker.internal 解析不了、Target 会一直 down）：
#   extra_hosts:
#     - "host.docker.internal:host-gateway"
$ docker compose up -d prometheus
→ 打开 http://localhost:9090 → Status → Targets，应该看到 gpu 也是 UP
```

解读：**Targets 里多出一个 UP 的 job，就说明 GPU 指标进来了**（这一步不会，回去看 5-02 的 Targets 那一节）。

步骤 3：在 Grafana 上画出显存曲线

```
Grafana → New panel → 查询框输入：

  100 * DCGM_FI_DEV_FB_USED / (DCGM_FI_DEV_FB_USED + DCGM_FI_DEV_FB_FREE)

标题改成"GPU 显存使用率" → Save
# 再配一条：DCGM_FI_DEV_GPU_UTIL → 标题"GPU 利用率"
```

解读：这条 PromQL 和 5-04 学的磁盘使用率**是同一种句型**（用了多少 ÷ 总量 × 100）—— 只是把 df 的数字换成了显存的数字。

步骤 4：造一次负载，看曲线动

```
$ python3 -c "
import torch, time
x = torch.randn(6000, 6000, device='cuda')
t0 = time.time()
while time.time() - t0 < 60:
    x = x @ x
"
# 回 Grafana：GPU 利用率飙高、显存曲线有一块台阶 —— 截图存档
```

完成判定

□ 能说出为什么 node-exporter 看不到 GPU、该用谁　□ Prometheus 的 Targets 里有 GPU job 并且是 UP　□ Grafana 上有"GPU 显存使用率"和"GPU 利用率"两块面板　□ 能说出这两条 PromQL 与 5-04 那条是同一个句型　□ 有一段负载下的曲线截图

回滚方式：`docker rm -f dcgm` 删掉 exporter；把 prometheus.yml 里 gpu 那段删掉再 `docker compose up -d` 即回到原样。

实战价值：把"我会监控"从 CPU/内存升级到 **GPU 可视化**——这正好是 AI 运维岗区别于普通运维的那一块。配上曲线截图，说服力很强。

8-09 读 DeepSeek-V3 技术报告，写 MoE / MLA 中文笔记

（参考：DeepSeek-V3 Technical Report，arxiv.org/abs/2412.19437。**不读原文也能完成本关**：用下面的大白话小节 + 8-06 的命令行助手提问就能把笔记写完。）

这一关不是让你啃论文，而是回答"聊聊你对 AGI 的理解"。你只需要**用自己的话讲清两个词**，再补一点自己的疑问。

步骤 1：先建立"大模型很贵"的直觉

```
模型越大 → 参数越多 → 显存越吃紧、算得越慢、花钱越多
于是大家想两件事：
  ① 怎么让**参数多但每次只用一部分**？ → 这就是 MoE
  ② 怎么让**长文本不要太占显存**？     → 这就是 MLA
```

解读：先有这个直觉，再看术语就不会晕 —— **MoE 和 MLA 都是"省"的招数**，一个省算力，一个省显存。

步骤 2：两个词的大白话版

词	大白话	为什么 AI 运维要懂
MoE （混合专家）	模型里有很多"专家小组"，来一个问题**只叫相关的几个专家**干活，其他不动 —— 参数量可以做到很大，但每次只跑一小部分	它决定了**显存占用和显存分布**：专家要分散在不同卡上，专家之间的调度和通信就是运维要盯的东西

词	大白话	为什么 AI 运维要懂
MLA（多头潜在注意力）	把"注意力"要记的东西**压缩后再存**，同样长的上下文，占的显存小很多	它直接影响**长文本能不能跑得起来**：同样的卡，能不能开更长的上下文

遇到不懂的词（KV cache、注意力、token…）**别去啃整篇论文** —— 用 8-06 的那个命令行助手问一句"用大白话解释 KV cache 是什么"，然后记下来。

步骤 3：写一页笔记（产出）

```
# DeepSeek-V3 技术报告读书笔记
1）它想解决什么：参数规模大 → 训练和推理都贵
2）MoE 用它干什么：参数多、但每次只激活一部分（省钱、省算力）
3）MLA 用它干什么：压缩注意力的存储（省显存、能开更长上下文）
4）和运维的关系：专家怎么分布到多卡、显存怎么切、通信怎么不拖后腿
5）我自己的疑问（至少一条）：比如"专家分布不均会不会导致某张卡特别忙？"
—— 疑问比结论更值钱，聊起来最有料的就是它
```

完成判定

□ 能用大白话讲清 MoE 和 MLA 各解决什么问题（不背定义）　□ 能说出它们和"显存 / 卡 / 通信"的关系
□ 有一页中文笔记（含至少一条自己的疑问）　□ 至少用命令行助手查过两个不懂的词

回滚方式：纯阅读与笔记，无风险。

实战价值：工作里聊到 AGI 理解，靠的就是这个。能说"我读过 DeepSeek-V3 的报告，我理解 MoE 是让参数大但每次只用一部分，MLA 是省显存；我有个疑问是专家分布不均会不会拖慢训练"——**带着疑问的答案，比背书好听得多。**

8-10 画一张全景图，讲 10 分钟（★）

为什么这一关是 ★：这类岗位的要求里写着"脑子里要有从 GPU 计算 → 显存 → NVLink → 网卡 → 存储 → 调度的完整地图"。**这不是加分项，是入场要求。**这一关就是把这十关学的东西压缩成一张图，练到能讲 10 分钟。

步骤 1：照这六层画（一层一行，别抄术语，用你自己的话）

```
① GPU 计算        —— 卡在不在干活？（nvidia-smi：GPU-Util；工具：8-03）
② 显存            —— 装得下吗？（FB_USED/FB_FREE；故障：OOM，8-04）
③ NVLink / 卡间   —— 多卡之间怎么快速换数据（同一台机器内的"高速通道"）
④ 网卡 / 网络     —— 多机之间怎么同步（NCCL 走网络；抖动会让所有卡一起等）
⑤ 存储            —— 数据和 checkpoint 从哪来、写多快（写 checkpoint 会抢带宽）
⑥ 调度            —— 谁来分配这些卡和资源（K8s 的 requests/limits，6-07）
```

画法建议：一张 A4 横放，左边画这条"从算力到调度"的竖线，右边每一层挂上：**① 看什么指标 ② 出问题什么样 ③ 我练过哪一关。**手画比打字好（讲的时候指着说）。

步骤 2：每层准备一个"这里会出什么问题"

```
① GPU 计算：利用率常年个位数 → 在等数据（查 DataLoader / CPU）
② 显存：    OOM → 尺寸/batch/碎片/泄漏四类病因（8-04 的记录直接搬过来）
③ NVLink：  某张卡慢 → 所有卡跟着等（木桶效应）
④ 网络：    训练一步忽快忽慢 → 通信延迟抖动（NCCL 日志）
⑤ 存储：    每存一次 checkpoint 就卡一下 → 存储带宽被抢
⑥ 调度：    Pod 一直 Pending → requests 要得比节点能给的多（6-07 见过）
```

步骤 3：录音讲一遍（10 分钟，对着图不看稿）

```
讲法骨架（照这个顺序说，10 分钟足够）：
1) 一句话总起：AI 运维盯的是一条从算力到调度的链子（30 秒）
2) 从上往下走六层，每层讲"看什么指标 + 出问题什么样 + 我练过什么"（6 分钟）
3) 举一个真实例子：显存 OOM 我复现过，是这样查的（2 分钟）
4) 我的疑问 / 想深入的方向（1 分钟）
— 录音存进 learning-journey，方便以后对比进步
```

完成判定

□ 有一张手画的全景图（六层齐全） □ 每层能说出一个"会出什么问题"和一个"看什么指标" □ 能对着图不卡壳讲满 10 分钟（录音为证） □ 至少能回溯到三个自己练过的关卡（点名编号）

回滚方式：无操作。

实战价值：这是阶段 8 的收官，也是工作里聊集群时的底稿。这类岗位要的就是这张图 —— 你不但有，还能讲。

阶段 8 • 学习总结练习：一张全景图 + 一次显存故障的完整交代

业务场景：组长（或未来的同事）问你："你说你懂 GPU，那我给你一张卡，你怎么知道它好不好？出了显存问题你怎么查？" —— 你要在 **10 分钟**里把这两个问题讲完，并且手里有实物。

综合要求：① 用 Ollama 起一个小模型并能调它（证明"模型跑得起来"）；② 在 Colab 或 Kaggle 上跑一次 `nvidia-smi` 并截图（证明"我见过真卡"）；③ **亲手复现一次 CUDA out of memory**，把报错原文和排查记录留下来；④ 把 GPU 指标接进监控面板（显存 / 利用率曲线截图）；⑤ 画一张六层全景图（GPU 计算 → 显存 → NVLink → 网卡 → 存储 → 调度），每层写"看什么指标 / 会出什么问题"；⑥ 录一段 10 分钟讲述。

验收：□ Ollama 能对话并有一条 curl 调用记录 □ nvidia-smi 截图（能看到型号/显存/利用率） □ OOM 报错原文 + 四类病因的排查记录 □ 监控面板上的显存曲线截图（没有 N 卡就按 8-08 的「降级演练」处理并标注"环境限制"） □ 六层全景图（手画）+ 10 分钟录音 □ 能说出"训练变慢了先看哪一层"

产出：仓库 `ai-inference-notes`（OOM 排查记录 + 全景图 + 讲述稿 + 截图）

AI 运维指向：这类岗位要的就是这张"从算力到调度"的地图；显存故障是 AI 运维日常的第一大类。

实战价值：“陌生故障”和“AGI / 集群理解”两类问题的合体准备。有实物、有录音、有图，讲起来不心虚。